

Name: _____

Collaborators: _____

THAYER SCHOOL OF ENGINEERING • DARTMOUTH

ENGS 28, W26

LAB 5 — TEMPERATURE CONTROLLER

Due: Tuesday February 10, 2026 2:15 PM

Contents

1 Overview	2
1.1 Place in course	2
1.2 Learning Objectives	2
2 Design Challenge	2
3 Specifications	2
3.1 Paper Design	2
3.2 ADC Device Driver	4
3.2.1 ADC with timer interrupt	4
3.3 Seven-Segment Display Driver	4
3.4 Build and Test	5
4 Additional Features	5
5 Postlab	7
A Tips from the past!	8

Deliverables

1 Paper Design (4 points)	3
2 ADC Driver (4 points)	4
3 Seven Segment Driver (4 points)	4
4 Finished Product (4 points)	5
5 Additional Feature 1 (1 point)	6
6 Additional Feature 2 (1 point)	6
7 Additional Feature 3 (1 point)	7
8 Additional Feature 4 (1 point)	7
9 Code Quality (5 points)	7

1 Overview

1.1 Place in course

You have now learned to use interrupts to detect button presses and have started looking at timer interrupts. You have also used the ADC by polling the `ADC_ISR_EOC` bit.

This lab will give you practice using interrupts to control your program rather than always polling from the main loop. This is a technique that is very common in embedded systems.

1.2 Learning Objectives

- Develop a device driver.
- Use a timer interrupt to control data acquisition and display.
- Develop a system design on paper and translate it step-wise to hardware and software.

2 Design Challenge

A customer has asked you to develop a thermostat interface and sensor system for a climate control unit that displays ambient temperature in degrees Celsius on a seven-segment display, updating once a second. Your task in this lab is to develop a working prototype.

3 Specifications

To complete this assignment, you must meet the following specifications (more details below):

1. You must develop a paper design (bill of materials, schematic/block diagram, firmware plan, state diagram if appropriate).
2. Develop a modular device driver for the analog to digital converter (ADC).
3. Enable the ADC with a timer interrupt rather than `delay_ms()`.
4. Develop a modular device driver for the seven-segment display
5. Demonstrate your system to an instructor or TA and get a signature.

The client has requested agnostic device drivers, so the seven-segment display and the ADC can be used in future projects. Further, they request that your code is well documented and that your paper designs are neat and easy to read.

If you complete this design challenge, you have met the base requirements for this assignment and will be awarded 16/20 points .

Note!

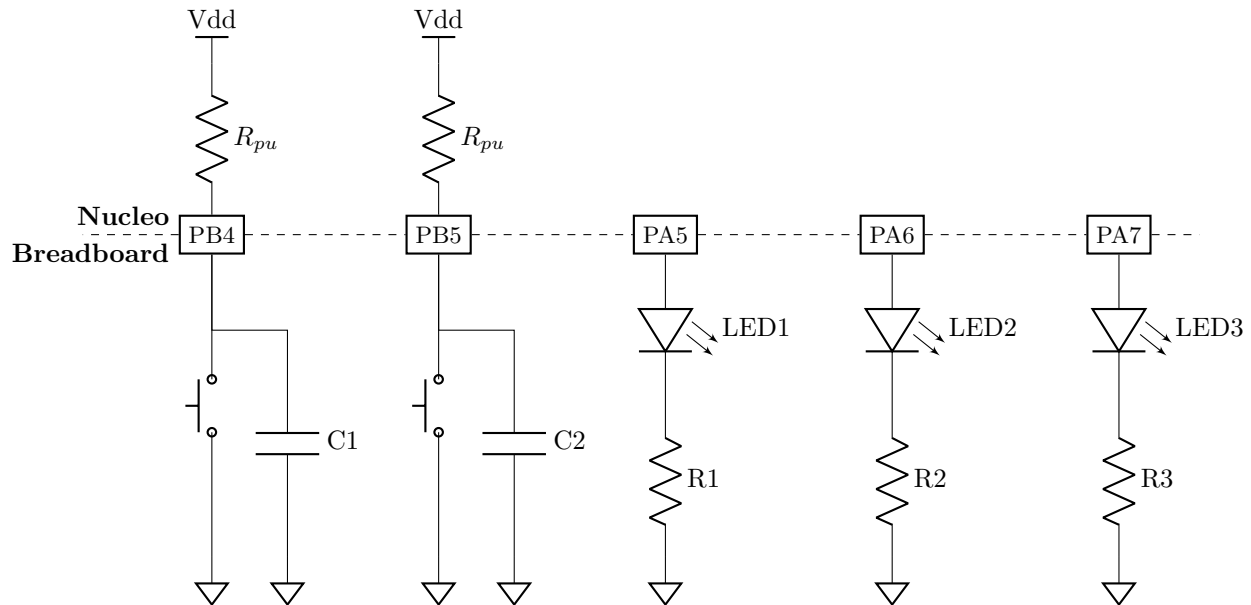
Do not attempt to implement this all at once. Build and test in incremental changes to your code. When you have something that works, save a copy.

3.1 Paper Design

All good engineering designs begin on paper. Paper design allows you to organize your ideas and to strive for clarity of thought. Begin by considering the components you will need in your design. Make a list called a “Bill of Materials” (BOM) that includes the name of the component, the quantity, and where applicable, the value. The BOM for the pushbutton game in Lab 2 might look like the following:

Bom		
Part	Value	Quantity
Nucleo		1
Pushbutton Swtich		2
LED (Red)		2
LED (Green)		1
Debouncing Capacitor	$0.15\mu F$	2
Current Limiting Resistor	330Ω	3

Once you have decided on your component selection, you will need to draw a circuit schematic. It is good practice to draw schematics with straight lines. A neat schematic for Lab 2 might look like the following:



Notice that every component in the BOM shows up in the circuit schematic. If you plan on adding in the additional features, leave room in your initial schematic to add them.

Investing time at the start of a project or lab for pin planning and drawing a circuit schematic will prove useful, particularly as you begin to include more elements on your breadboard. In addition to enabling clarity of thought when planning, schematics are a useful tool for communicating your ideas to other engineers and will be invaluable to you as you debug your breadboard.

Please note that the paper design is not just for circuit schematics. State machines and pseudocode can be incredibly valuable tools for designing code. More detail on paper translates directly to less time coding and less time debugging!

Important!

You never want to design in code—rather, use code to translate a paper design into a prototype.

Deliverable 1: Paper Design (4 points)

Submit your paper design as described above, with the rest of your submission

3.2 ADC Device Driver

A device driver is a self-contained, modular library that contains key functions used to operate the device it “drives” or runs. These libraries are written to be easy to incorporate into any project, and you should strive to do so here (you will be using the ADC throughout the rest of this course, so this will save you time in later labs!).

Write two files, `ADC.c` and `ADC.h`, placing them in **Library**. Add `#include <ADC.h>` in your main program.

Your library will consist of four functions. Here are the prototypes that will be in `ADC.h`:

```
void adc_init( void );           // Initialize the ADC
void adc_setChannel( unsigned int chNum ); // Change to a new channel
void adc_setWidth( int widthCode ); // Set the precision
int adc_getValue( void );       // Start a conversion and return its result
```

`ADC_getValue()` is identical to what you used last week (but renamed for consistency). `ADC_setChannel()` allows the user to select a different analog input channel. `ADC_setWidth()` allows the user to select the data width (precision) returned by the ADC. `ADC_init()` has the same functionality as before, but you should rewrite it to use your new functions `ADC_setChannel()` and `ADC_setWidth()`.

Make other necessary modifications to your `Lab5.c` (or whatever you call your main program). Call `ADC_setWidth()` to produce a 12 bit result.

The best way to test your library is to use the potentiometer to supply an input voltage rather than the temperature sensor. Once you know it’s working, change the ADC input back to the TMP235 and make sure that you’re getting correct temperatures.

Ensure that your device driver is well commented, so the client can easily understand how it works (or you, three weeks from now, as you look back at this driver).

3.2.1 ADC with timer interrupt

As you saw in class, a timer interrupt is superior to `delay_ms()` for making things happen at regular intervals. Following the steps in the lecture notes, modify your code to use the timer interrupt, and verify its correct operation.

Deliverable 2: ADC Driver (4 points)

Submit a well commented pair of files (`.c` and `.h` that implement the ADC driver as defined above.

3.3 Seven-Segment Display Driver

Like the timer interrupt, use the code you developed in class on Day 10 as a basis for your seven-segment display device driver.

As a start, your header file for the 7 segment driver should declare the following functions:

```
#define HT16K33_NBUF          5           // Size of display buffer

// Function prototypes
void SevenSeg_init(void);               // Initialize the display
void SevenSeg_blink(uint8_t rate);      // Blink the display
void SevenSeg_dim(uint8_t brightness);   // Dim the display

// display_buffer is expected to be of size HT16K33_NBUF
void SevenSeg_write(uint16_t *display_buffer); // Write the display RAM
```

Deliverable 3: Seven Segment Driver (4 points)

Submit a well commented pair of files (`.c` and `.h` that implement the Seven Segment driver as defined above.

3.4 Build and Test

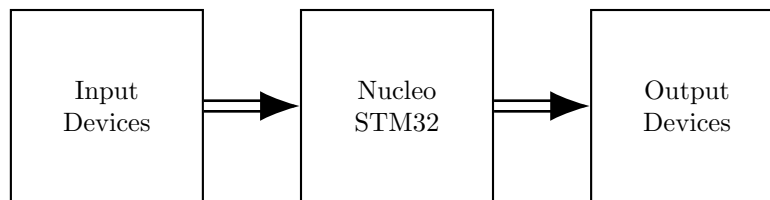
Assemble the individual pieces described above into the full system as specified in Section 3

This is your first large design—this is a complicated lab and there are many (figuratively speaking) moving parts! Any time you work on a complicated project, you want to break it down into manageable sections. Do not try to implement everything all at once and hope that it works! You want to make your design process robust to mistakes, because as people you are prone to making them. That is ok! Making a mistake does not mean you are a bad embedded systems designer. There is no ego here. You would rather anticipate that you will make mistakes and have steps in your design and fabrication process that are designed to catch them.

Take the big, complicated project and break it down into smaller problems that are more manageable to solve. Then, solve the easier problems one at a time rather than trying to tackle the whole hard problem. If you do this correctly, solving all the easy problems solves the hard problem.

For example, in this lab, you might start with a single function in your ADC driver library and get that to work. Then add in another function and test that. Once you have that working, add another function, test it, and so forth. Once you have your ADC driver library working in full, make sure you save a copy to refer to later. Do this for each stage of this lab.

If you are debugging a problem when you have most of your system built up, recall that the signal flow in your embedded system is as follows:



If an aspect of your circuit is not working properly, try testing it in isolation. If it is not easy to isolate, start at the left of the above diagram and work your way node by node through the circuit to the right, until you find the location where something is going wrong. Frequently, the fix is trivial and identifying the location of a bug is the challenging part.

Deliverable 4: Finished Product (4 points)

Demonstrate your completed project to a member of staff.

Staff Signature _____

4 Additional Features

More points will be awarded for delivering additional features, as described below, for 1 point each. Demonstrate any of these to a TA or instructor and get signature.

1. When the ambient temperature is below a threshold called a set point, a green LED is illuminated, indicating that everything is fine. When the ambient temperature rises above the set point, a red LED flashes at 1 Hz, indicating that everything is NOT fine. The green LED turns off. The red LEDs on time should be controlled with a timer.
2. Control the set-point with the potentiometer and include hysteresis in the behavior of the LEDs. Once the temperature has risen above the set point, the red light does not turn off and the green light does not turn on again until the temperature has fallen below the set point by a certain amount. This prevents the LEDs from flickering if the temperature is right at the set point.

3. Modify the 7-segment display driver to include a “C” character for readings in Celsius and an “F” character for readings in Fahrenheit. Control the display with a push button.
4. Finally, you may propose and implement an additional feature (or appropriate easter egg) for the customer. Be creative and challenge yourself!

Deliverable 5: Additional Feature 1 *(1 point)*

Demonstrate your additional feature.

Brief Description: _____

Staff Signature _____

Deliverable 6: Additional Feature 2 *(1 point)*

Demonstrate your additional feature.

Brief Description: _____

Staff Signature _____

Deliverable 7: Additional Feature 3 (*1 point*)

Demonstrate your additional feature.

Brief Description: _____

Staff Signature _____

Deliverable 8: Additional Feature 4 (*1 point*)

Demonstrate your additional feature.

Brief Description: _____

Staff Signature _____

Deliverable 9: Code Quality (*5 points*)

Before submitting your code, review it to ensure that it has all of the following features (as mentioned in Lecture)

1. Style (properly indented, readable, well organized)
2. Commenting
3. Use of bitmasks
4. Modularity
5. Clarity
6. Efficiency

5 Postlab

Over the last two weeks, you have developed quite a bit of code (and understanding) of the ADC and the Seven Segment display. If you wish to reuse it, now is a good time to move the `.c` and `.h` files into your library folder. That way, they will be there when you need them next time!

A Tips from the past!

Note!

Ben was the lab instructor early in the history of this course. His excellent advice for debugging is included here.

- Always go into a project or lab with a plan. If something is not clear to you, return to the paper design until it is clear. Clarity of thought is paramount.
- Do not debug when you are tired or frustrated. You CAN lose progress on a circuit in this class if fatigue clouds your judgement or your ability to see clearly.
- Take frequent breaks, and do not be afraid to step away from a problem if you are struggling with it. Often you can solve the problem quickly when returning to it later, having given yourself time to mull over the problem while doing other things.
- DO NOT work on a problem for eight hours straight. That is not a good use of your time. Start early so you won't feel a time crunch later. It is generally counterproductive to stay up all night to finish an assignment.
- Read through the lab handout when you get it so you know what to expect. What you do not know will hurt you. Allocating the right amount of time for a project can be challenging, as you don't know how long it will take to debug issues that will inevitably arise.
- Document as you go for two reasons. 1. You will be spared the documentation crunch at the end of a project. 2. You will be using your ADC driver in three weeks and will want to understand how it works. Documenting spares you or a colleague a future pain. Never retrofit documentation.
- Take care of yourself. Make sure that you are investing time in getting sleep, exercise, good nutrition, hydration, and time to rest and pursue hobbies you enjoy outside of class. This will make you a better embedded systems designer.
- Taken together, this advice may make working on a project feel slow. I assure you, it will save you time in the end.